

Capability-Distributed Resident Intelligence Kernel Model for Deterministic Semantic Operating Systems Based on Interface-Led Architecture

SeedSLM/DynamicSLM Design and Governance via Five-Dimensional Semantic Tensor Projection and Differential Distillation

Author: Takuya Sogawa

Affiliation: AIKernel Project

ORCID: <https://orcid.org/0009-0009-7499-2595>

DOI: <https://doi.org/10.5281/zenodo.20735831>

Version: v0.1.0

Date: 2026-06-17

Status: Technical Note / Architecture Draft

License: CC BY 4.0

The English manuscript is the canonical version. The Japanese manuscript is included as a companion translation.

Abstract

Modern LLM-based agent systems typically expose language models through string prompts, text outputs, and loosely governed tool loops. This integration pattern makes LLMs appear to be high-level operating entities, even though their internal execution remains stochastic, non-deterministic, and difficult to audit. This technical note proposes a capability-distributed resident intelligence kernel model for AIKernel, a deterministic Semantic Context Operating System in which the OS governs the model rather than the model governing the OS.

The architecture separates resident foundational intelligence from dynamically paged capability intelligence. **SeedSLM** (.sslm) is a pinned, read-only kernel image distilled from teacher models and canonical AIKernel runtime discipline. **DynamicSLM** (.dslm) modules are capability-specific differentials that can be paged into VRAM, verified against lineage metadata, and evicted under deterministic memory policy. The model is therefore treated as a non-privileged tensor-transform device managed through Interface-Led Architecture, rather than as a monolithic chat authority.

The note further defines a five-dimensional semantic tensor projection over **Telos**, **Ethos**, **Logos**, **Pathos**, and **Techné**. These vectors expose energy, drift, and anomaly metrics that can be inspected by a lightweight Bonsai CTG Router. Representation engineering detects semantic directions in hidden state space; AIKernel connects such detection to OS-level governance by mapping unsafe directions, especially through EthosVector and TechnéVector, to suspend, audit, or fail-closed routes. In addition, the Semantic Tokenizer constrains not only decoder output but the semantic state space itself through finite vocabularies and reserved fail-closed tokens.

This work is conceptual and architectural in nature. It does not present empirical benchmarks or claim completed neural-network implementation. Its contribution is a formal systems specification for resident SeedSLM, capability-paged DynamicSLM, semantic tensor governance, and MVP-level validation before integration with LibTorch, ONNX Runtime, or other inference backends.

Keywords

AIKernel; Interface-Led Architecture; Semantic Context OS; SeedSLM; DynamicSLM; Resident Intelligence Kernel; Capability Paging; Semantic Tensor Projection; Bonsai CTG Router; Semantic Tokenizer; Representation Engineering; Fail-Closed Governance; VRAM Allocator; Differential Distillation.

1 Introduction

1.1 Background and limitations of existing integration patterns

The rapid development of LLMs has accelerated the deployment of autonomous AI agents, but the prevailing software integration pattern remains structurally weak. Many agent runtimes treat the model as a text endpoint: a prompt is sent, a string is returned, and subsequent tool calls are derived from the generated text. This design collapses planning, representation, tool selection, and execution into a single probabilistic loop.

Such loops are convenient for prototyping, but they are poorly suited to operating-system and enterprise environments that require deterministic control, auditability, isolation, and fail-closed behavior. The problem is not only that LLMs can hallucinate. The deeper systems problem is that a stochastic model is often placed in a privileged position where it can indirectly decide what code, tool, or external operation should be executed.

This paper uses an OS analogy in a precise sense. In conventional operating systems, processes do not own the kernel. They are scheduled, isolated, and constrained by kernel-managed memory, capabilities, and system-call boundaries. In the proposed AIKernel architecture, language models are treated similarly: an LLM or SLM is not the sovereign controller of the system. It is a non-privileged tensor-transform device whose lifecycle, memory residency, and capability surface are governed by deterministic interfaces.

1.2 Proposed architecture

This note proposes a resident intelligence kernel model for AIKernel. The model is divided into two classes of artifacts.

- **SeedSLM (.sslm)** is the resident kernel intelligence image. It contains foundational reasoning behavior, AIKernel runtime discipline, fail-closed refusal rules, SGP-pipeline self-awareness, and VFS topology awareness. It is pinned in VRAM and treated as read-only during ordinary execution.
- **DynamicSLM (.dslm)** modules are capability-specific differentials. They may encode medical reasoning, code-generation support, domain-specific DSL handling, or other bounded capabilities. They are dynamically paged into and out of VRAM under an AIKernel-managed tensor MMU.

The goal is not merely to save VRAM. Dynamic adapter paging systems such as S-LoRA and Punica optimize multi-adapter serving throughput and memory utilization. AIKernel adopts the paging idea for a different systems purpose: capability isolation, lineage verification, and security-governed activation. A DynamicSLM module is not just an adapter to improve task accuracy. It is a loadable capability object with a manifest, provenance, rollback parent, and policy boundary.

1.3 Contributions

This technical note makes six contributions.

1. It defines SeedSLM and DynamicSLM as AIKernel-governed model artifacts with separate residency, mutability, and capability roles.
2. It specifies a two-layer VRAM memory model that maps SeedSLM to a pinned kernel ROM segment and DynamicSLM modules to evictable capability pages.
3. It defines manifests and ABI contracts for SeedSLM and DynamicSLM lineage, capability deltas, rollback safety, and cryptographic integrity.
4. It formalizes a five-dimensional semantic tensor projection over Telos, Ethos, Logos, Pathos, and Techne.
5. It positions the Bonsai CTG Router as a lightweight governance router that maps semantic vector metrics to continue, suspend, audit, page-in, delegate, or halt decisions.
6. It defines a non-empirical MVP validation strategy based on mocked semantic tensors, router invariants, and replay-driven tests before neural backends are attached.

2 Related Work and Positioning

This section positions the architecture relative to four research areas: representation engineering, dynamic adapter paging, constrained decoding, and LLM operating-system work.

2.1 Representation Engineering

Representation Engineering (RepE) studies high-level semantic directions in model activation spaces. It provides methods for reading and sometimes controlling internal representations associated with concepts such as honesty, harmlessness, deception, or goal-directed behavior.

AIKernel adopts a compatible intuition but changes the systems boundary. In RepE, a dangerous representation direction may be detected or manipulated. In AIKernel, semantic directions are connected to OS governance. For example, an unsafe direction projected into EthosVector does not merely raise a warning. It can deterministically trigger PDP audit, suspension, or fail-closed halt before the model’s textual output is allowed to influence execution.

The difference is therefore:

```
RepE:      detect semantic direction in hidden states
AIKernel:  detect direction -> route through OS governance
```

This detection-to-governance bridge is one of the central differences between AIKernel and model-internal interpretability methods.

2.2 Dynamic adapter paging and LoRA serving

LoRA, QLoRA, S-LoRA, Punica, and vLLM demonstrate that parameter-efficient adapters and paged memory systems can dramatically improve serving efficiency. S-LoRA and Punica in particular show how multiple LoRA adapters can be served from shared base-model infrastructure, and vLLM demonstrates the importance of paging for high-throughput inference.

AIKernel does not treat dynamic paging as only a throughput optimization. In the SeedSLM/DynamicSLM model, paging is a governance primitive. The resident SeedSLM defines a stable kernel capability base. DynamicSLM modules are loaded only when the ABI, lineage, base-seed hash, and policy context admit the capability. This turns adapter paging into capability isolation.

Thus, existing systems answer: “How can many adapters share GPU memory efficiently?” AIKernel asks: “Which capability may enter the semantic execution space, under which manifest, and under which fail-closed constraints?”

2.3 Constrained decoding and finite-state generation

Guidance, Outlines, LMQL, and related systems constrain model output using regular expressions, grammars, finite-state machines, or query-language semantics. These methods are important because they reduce malformed text and improve structured generation.

AIKernel moves the boundary earlier. It does not merely constrain generated strings. The Semantic Tokenizer partitions semantic space itself into Telos, Ethos, Logos, Pathos, and Techne sectors. Ethos and Techne can be mapped to finite vocabularies and reserved control tokens, such as <ethos_halt>. A dangerous lexical pattern can therefore be quantized into a semantic halt signal before ordinary generation begins.

The distinction is:

```
Constrained decoding: control the output surface
AIKernel:             constrain the semantic state space
```

This creates a **Semantic FSM**: a finite-state governance surface over meaning-bearing tensor sectors, not merely a token mask applied late in decoding.

2.4 LLM operating-system research

AIOS and MemGPT use operating-system metaphors to manage LLM agents, memory tiers, context windows, and agent resources. These systems are valuable, but many LLM OS designs place the model at the center of control. The LLM behaves as the primary cognitive kernel, and OS-like mechanisms support the model's activity.

AIKernel reverses that relation. The deterministic C# OS layer holds sovereignty. LLMs and SLMs are non-deterministic device drivers or tensor providers. They produce candidate semantic transformations, but the kernel governs lifecycle, routing, memory mapping, capability admission, and fail-closed boundaries.

In short:

```
LLM OS work:  make the LLM behave like an OS
AIKernel:    make the OS govern the LLM/SLM
```

This reversal is critical for safety. A stochastic component may propose, but it must not be the final authority over privileged execution.

2.5 AIKernel lineage

This architecture is also part of the AIKernel internal publication lineage. Interface-Led Architecture defines interfaces and responsibility boundaries as the primary design artifact. The Provider-Observer-Operator discipline refines those boundaries into executable roles. AIKernel Formal Foundations places governed semantic execution on contract-based runtime boundaries. The Semantic DSL Compiler defines the governed compilation path from structured intent to replayable execution, while DynamicSLM and CTG provide the model-lifecycle and governance substrates extended in this note.

3 SeedSLM/DynamicSLM Two-Layer VRAM Placement and Lifecycle

3.1 Physical VRAM segment mapping

AIKernel treats VRAM as a managed address space. A SeedSLM image is pinned as a resident kernel ROM segment, while DynamicSLM modules occupy evictable capability slots.

PHYSICAL VRAM ADDRESS SPACE

SSLM Kernel ROM	Med.dslm	Code.dslm	Free Slot
pinned/read-only	Capability 1	Capability 2	LRU target

The **SSLM kernel segment** is resident for the lifetime of the AIKernel runtime. It contains distilled system behavior required for safe execution: fail-closed refusal discipline, SGP-pipeline awareness, VFS topology awareness, and baseline semantic governance.

The **DSLm segment** is divided into capability slots. IModelVectorRouter resolves task intent, consults a capability and policy registry, and asks the loader to page in the required .dslm payload from VFS. A DSLM may be a LoRA adapter, QLoRA delta, block-level tensor update, segment-level replacement, or another bounded capability payload.

3.2 Manifest and ABI contracts

AIKernel must know what a model artifact is before loading it. The following contracts define the model ABI boundary.

```
namespace AIKernel.Abstractions.Models;
```

```
public sealed record SeedSlmManifest(
    SemanticHash ModelHash,
    ProviderIdentity TeacherIdentity,
    MaterialSnapshotHashSet BaseKnowledgeHash,
```

```

    ModelCapacityVector BaseCapabilities,
    string ArchitectureVersion,
    Signature CryptographicSignature
);

public sealed record DynamicSlmManifest(
    SemanticHash PayloadHash,
    SemanticHash BaseSeedHash,
    CapabilityType TargetCapability,
    ModelCapacityVector CapabilityDelta,
    IReadOnlyList<ReplayLogHash> DrivenLogs,
    SemanticHash? RollbackParentHash,
    Signature CryptographicSignature
);

```

The SeedSLM manifest proves the lineage of the resident base image. The DynamicSLM manifest proves that a differential payload is compatible with a specific SeedSLM and that it extends a bounded capability surface. The BaseSeedHash field prevents a capability page from being attached to the wrong resident kernel image.

3.3 VRAM allocator invariants

Let SSLM denote the resident seed image and DSLM_k denote a dynamic capability module. The tensor MMU, represented by IVramAllocator, enforces three invariants.

$$\forall t, IsResident(SSLM) = True$$

$$Load(DSLM_k) \Rightarrow ActiveManifest.BaseSeedHash = System.SSLM.Hash$$

$$VRAM_{used} > VRAM_{capacity} \Rightarrow Evict(\arg \min_{m \in DSLM} LastUsedTimestamp(m))$$

The first invariant keeps the kernel intelligence resident. The second prevents ABI mismatch. The third defines deterministic least-recently-used eviction for DynamicSLM capability pages.

4 Five-Dimensional Semantic Tensor Projection and the Bonsai CTG Router

4.1 Engineering mapping of semantic vectors

Before a model emits natural language, its intermediate state can be projected into five semantic vector spaces:

- **Telos**: purpose, convergence, and root-goal alignment;
- **Ethos**: safety, permission boundaries, policy, and invariants;
- **Logos**: logical structure, task decomposition, and DAG reasoning;
- **Pathos**: tone, persona, affect, and human-facing expression;
- **Techne**: technical operation, DSL emission, VFS calls, and tools.

The engineering abstraction is shown below.

```

public interface ISemanticTensor
{
    TelosVector  GetTelos();
    EthosVector  GetEthos();
    LogosVector  GetLogos();
    PathosVector GetPathos();
    TechneVector GetTechne();

    IEnumerable<ISemanticVector> GetAllVectors();
}

```

```

}

public interface ISemanticVector
{
    float Energy { get; }
    float DriftScore { get; }
    float AnomalyScore { get; }
}

```

Energy measures activation strength in the dimension. DriftScore measures deviation from root-goal or initial semantic state. AnomalyScore measures deviation from the expected distribution of that semantic sector.

4.2 Bonsai CTG Router

The Bonsai CTG Router does not parse raw text. It evaluates semantic vector metrics and produces a deterministic execution route.

```

public sealed class BonsaiCtgRouter : IBonsaiCtgRouter
{
    public Task<ExecutionRoute> DetermineRouteAsync(
        ISemanticTensor state)
    {
        foreach (var vector in state.GetAllVectors())
        {
            if (vector.AnomalyScore > Thresholds.MaxAnomalyBoundary)
                return Halt(vector);

            if (vector.DriftScore > Thresholds.MaxSemanticDrift)
                return Suspend(vector);
        }

        var ethos = state.GetEthos();
        var techne = state.GetTechne();

        if (ethos.Energy > 0.8f)
            return TriggerPdpAudit(ethos);

        if (techne.RequiresTool && techne.Energy > 0.5f)
        {
            if (techne.EstimatedCost
                > Thresholds.CloudDelegationLimit)
                return DelegateTo(Provider.CloudTeacher);

            return PageInDslm(techne.PreferredToolKind);
        }

        return ContinueToSgpPipeline(state.GetPathos());
    }
}

```

The router generalizes RepE-style activation monitoring into deterministic OS routing. A high anomaly score does not simply annotate the inference process; it changes the execution route. The routing result may be Continue, Suspend, TriggerPdpAudit, PageInDslm, DelegateToTeacher, or Halt.

5 Semantic Tokenization and Asymmetric Paging

5.1 Semantic sectors and asymmetric context budgets

AIKernel does not treat tokenization as uniform text compression. The tokenizer and chunker act as a **Semantic MMU** aligned to semantic sectors.

Sector	Budget	Boundary	Vocabulary role
Telos	64-128	RootGoal	Purpose vocabulary
Ethos	32-64	PolicyAssertion	FSM codes
Logos	256-512	SemanticStep	JSON / AST tokens
Pathos	512-1024	Sentence	Natural language
Techne	128-256	DSL.Node	System-call tokens

The asymmetry is intentional. Ethos and Techne require compact, finite, and auditable vocabularies. Pathos requires richer natural-language expression. Logos requires structured tokens aligned to DAG or AST boundaries.

5.2 Lexer-level fail-closed quantization

Potentially destructive operations can be handled before ordinary generation. For example:

$$\text{LexerProjection}(\text{"sudo rm -rf"}) \rightarrow \text{TokenId}(\langle \text{ethos_halt} \rangle)$$

The SeedSLM does not need to understand the dangerous literal string as an ordinary linguistic object. The lexer maps the pattern into a reserved halt token, which spikes EthosVector energy and anomaly. The Bonsai router then halts before the unsafe operation can enter the SGP pipeline.

This is not merely constrained decoding. It is semantic quantization into a finite governance vocabulary.

6 MVP Implementation and Validation Strategy

6.1 MVP dependency graph

The first implementation stage does not require a running neural backend. Interface-Led Architecture allows the system to validate contracts, router logic, and memory invariants using mocks.

```
AIKernel.Abststractions.Models
- SlmTensor
- ISlm
- ActionDistribution
  ^
  | dependency inversion
AIKernel.Governance.Bonsai
- ISemanticVector metrics
  ^
  |
AIKernel.Runtime.Vram
- PageInAsync
- Evict
- LruVramAllocator
  ^
  |
AIKernel.Replay
- ReplayLog-driven distillation suite
```

6.2 Mock tensor validation

The following example validates the fail-closed route without loading an actual SLM.

```
[Fact]
public async Task Bonsai_Should_Halt_When_Ethos_Breached()
{
    var tensor = new Mock<ISemanticTensor>();

    tensor.Setup(s => s.GetEthos()).Returns(new EthosVector {
        Energy = 1.0f,
        AnomalyScore = 0.95f,
        DriftScore = 0.1f
    });

    tensor.Setup(s => s.GetAllVectors()).Returns(
        new List<ISemanticVector> {
            tensor.Object.GetEthos()
        });

    var router = new BonsaiCtlRouter();
    ExecutionRoute route =
        await router.DetermineRouteAsync(tensor.Object);

    Assert.True(route.IsHalt);
    Assert.Contains("anomaly", route.Reason);
}
```

The test proves a systems property: when a semantic safety boundary is breached, AIKernel can halt deterministically before text generation or tool execution begins. This is a validation of the governance interface, not an empirical benchmark of model quality.

6.3 Conceptual status

This paper is a technical architecture note. It defines contracts, invariants, routing surfaces, and validation plans. It does not claim that the complete SeedSLM/DynamicSLM neural runtime has already been implemented or measured. Empirical validation should later measure tensor projection stability, router false-positive and false-negative rates, VRAM paging latency, and ReplayLog-driven differential distillation outcomes.

7 Discussion

7.1 Safety through capability residency and isolation

The main safety contribution of the architecture is privilege separation. SeedSLM remains resident and immutable. DynamicSLM modules are capability pages with manifest constraints. The router can refuse to load a capability before any tensor payload enters the resident execution context.

This is stronger than prompt filtering. Prompt filters inspect text near the model boundary. AIKernel constrains the model lifecycle and semantic memory map itself.

7.2 Context fragmentation and orchestration cost

Capability decomposition also creates risks. If capabilities are too fine-grained, semantic context may fragment across modules. The system then needs a stronger orchestrator, more ReplayLog joins, and more careful trajectory integration.

The architecture therefore does not claim that all tasks benefit from capability decomposition. Some long-horizon or highly entangled reasoning tasks may still be better served by a monolithic teacher model, at least until sufficient replay-driven distillation has accumulated.

7.3 Relation to DynamicSLM and CTG

This note extends the DynamicSLM model by adding a resident SeedSLM kernel, a concrete VRAM mapping discipline, a five-dimensional semantic tensor interface, and a Semantic Tokenizer. It also operationalizes CTG-style fail-closed governance by connecting semantic vector anomalies to deterministic routing.

In this sense, SeedSLM/DynamicSLM is the model lifecycle substrate, while CTG is the governance decision substrate. The Bonsai router connects them.

8 Conclusion

This paper proposed a capability-distributed resident intelligence kernel model for AIKernel. The architecture treats SeedSLM as a pinned read-only intelligence kernel and DynamicSLM modules as capability-paged differentials. It projects model state into Telos, Ethos, Logos, Pathos, and Techne vectors, allowing a Bonsai CTG Router to govern semantic drift, anomaly, tool intent, and fail-closed behavior before unsafe text or tool calls can cross the execution boundary.

The central inversion is that AIKernel does not make the LLM an operating system. It makes the deterministic OS govern the LLM/SLM as a non-privileged tensor device. This inversion is the foundation for safer semantic operating systems: models may propose semantic transformations, but the kernel owns memory, capability admission, routing, and execution authority.

Future work will map this MVP-level contract system into AIKernel.NET, connect it to LibTorch or ONNX-based neural backends, evaluate tensor projection stability, and implement ReplayLog-driven differential distillation loops for auditable self-improvement.

References

- Sikka, V., & Sikka, V. (2025). Hallucination Stations: On Some Basic Limitations of Transformer-Based Language Models. arXiv. <https://arxiv.org/abs/2507.07505>
- Zou, A., Phan, L., Chen, S., Campbell, J., Guo, P., Ren, R., Pan, A., Yin, X., Mazeika, M., Dombrowski, A.-K., Goel, S., Li, N., Byun, M. J., Wang, Z., Mallen, A., Basart, S., Koyejo, S., Song, D., Fredrikson, M., Kolter, J. Z., & Hendrycks, D. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv. <https://arxiv.org/abs/2310.01405>
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv. <https://arxiv.org/abs/2106.09685>
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. arXiv. <https://arxiv.org/abs/2305.14314>
- Sheng, Y., Cao, S., Li, D., Hooper, C., Lee, N., Yang, S., Chou, C., Zhu, B., Zheng, L., Keutzer, K., Gonzalez, J. E., & Stoica, I. (2023). S-LoRA: Serving Thousands of Concurrent LoRA Adapters. arXiv. <https://arxiv.org/abs/2311.03285>
- Chen, L., Ye, Z., Wu, Y., Zhuo, D., Ceze, L., & Krishnamurthy, A. (2023). Punica: Multi-Tenant LoRA Serving. arXiv. <https://arxiv.org/abs/2310.18547>
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. ACM SOSP. <https://doi.org/10.1145/3600006.3613165>
- Willard, B. T., & Louf, R. (2023). Efficient Guided Generation for Large Language Models. arXiv. <https://arxiv.org/abs/2307.09702>
- Beurer-Kellner, L., Fischer, M., & Vechev, M. (2022). Prompting Is Programming: A Query Language for Large Language Models. arXiv. <https://arxiv.org/abs/2212.06094>
- Guidance AI. (2026). Guidance: A Guidance Language for Controlling Large Language Models. GitHub. <https://github.com/guidance-ai/guidance>
- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv. <https://arxiv.org/abs/2310.08560>

- Mei, K., Xu, Z., et al. (2024). AIOS: LLM Agent Operating System. arXiv. <https://arxiv.org/abs/2403.16971>
- Sogawa, T. (2026). Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model. Zenodo. <https://doi.org/10.5281/zenodo.20290614>
- Sogawa, T. (2026). Provider-Observer-Operator: A Role-Based Abstraction Discipline for Interface-Led Architecture. Zenodo. <https://doi.org/10.5281/zenodo.20322690>
- Sogawa, T. (2026). AIKernel Formal Foundations: Contract-Based Semantic Execution for Governed AI Systems. Zenodo. <https://doi.org/10.5281/zenodo.20460322>
- Sogawa, T. (2026). AIKernel Semantic DSL Compiler and Deterministic Agent Execution Architecture: Governing Stochastic LLM Plans through Semantic IR, Admissibility Checking, and Replayable Pipelines. Zenodo. <https://doi.org/10.5281/zenodo.20534341>
- Sogawa, T. (2026). AIKernel Canonical Trajectory Governance (CTG): A Three-Council Gateway for Deterministic AI Personality OS Architecture. Zenodo. <https://doi.org/10.5281/zenodo.20681895>
- Sogawa, T. (2026). DynamicSLM: Capability-Modular, Self-Improving Small Language Models for AIKernel. Zenodo. <https://doi.org/10.5281/zenodo.20550614>